Test Project

Cloud Computing

JP-KR Samsung Skills Friendship Match

Day 4

Copied by. Worldskills Korea 2025 national competition - day2

© Samsung Electronics

Contents

This Test Project consists of the following documentation/files:

1. SFC2026_TP53_day1_actual_en
2. SFC2026_TP53_day2_actual_en
3. SFC2026_TP53_day3_actual_en
4. **SFC2026_TP53_day4_actual_en**
5. **Binaries in the s3 bucket**

Description of test project

Goals

This Test Project aims to rigorously evaluate your cloud computing skills through a series of challenges.

Background

You're a cloud engineer in the support department of enterprise company. You're tasked with solving problems arising across multiple projects and developing solutions by conducting small-scale proof-of-concept (PoC) projects to improve the enterprise system. You are given modules to solve in the enterprise, and you have to solve them within the given time!

Caution

1. Each module is independent and does not affect the others.
2. Each module must be implemented in a specified region.

Technical Details

Module 1: Implement CI/CD pipeline

Description

Create a CI/CD pipeline to reliably deploy and operate applications in a containerized environment. This automatically performs builds, tests, container image creation, and registry uploads whenever source code changes occur, ultimately automatically deploying to a container-based execution environment, efficiently managing the entire development-to-deployment process. Please solve this module using the **ap-southeast-1** region.

Tasks

1. ECR registry

To efficiently manage container images, create a container registry. Build a custom image based on nginx:latest , with the provided distribution file (index.html) set as the default page. Image tags are specified in the vX.X.X format (e.g., v1.0.0) to facilitate version management. It has to configure the ECR repository is named ***app-repo***.

2. EKS

We use EKS for efficient and easy container management. Application pods must run in the **"wsc2025"** namespace. The deployment name should be **"nginx-deploy"**. Before marking, create a pod using the image generated during the competition using the provided deployment file. Create resources by referring to the information below, and refer to best practices and well-architected frameworks for anything not specified.

© Samsung Electronics

- EKS Cluster Name: **wsc2025-cluster**
- EKS Node Instance Type: **t3.medium**
- EKS NodeGroup name: **wsc2025-app-ng**
- EKS Node Name: **wsc2025-app-node**

3. ELB

Provide endpoints that allow access to application Pods. These endpoints must be accessible from outside, and they must remain accessible even when Pods are restarted or added. Create resources by referring to the information below, and refer to best practices and well-architected frameworks for anything not specified.
- LoadBalancer Type: **ALB**
- LoadBalancer Name: **wsc2025-cicd-alb**

4. Github

Place the given deployment files, *index.html* and version files, at the top(root) of the repository, create Kubernetes manifest files in the *manifest/* path, and specify the name of the manifest that deploys the Deployment as "**deploy.yaml**". If the "index.html " file is changed or modified through Github Action, Github Action will automatically run and use the version file to specify a tag and create a new image. It will also change the image in "**deploy.yaml**" to the new image. However, Github Action should only run when *index.html* has changed. When marking, check whether all Github Action jobs are successful, and the time limit from commit to build completion is 5 minutes.
- Repository Name: **wsc2025-argocd-repo**
- Repository Type: **Public**

5. ArgoCD

When a new image is built via GitHub Action, Pods using the new image must be created within 5 minutes. The image must be automatically restored if an administrator changes it through any means other than code changes in the GitHub Repository. (For marking, the HTML page must load within 5 minutes after the GitHub Action job completes.)
- ArgoCD App Name: ***wsc2025-argo-app***

Module 2: Protect sensitive data in your s3 bucket

Description

To strengthen the security of data stored in Amazon S3, an object storage service, and prevent data leaks, we've implemented a data protection solution based on Amazon Macie. Leveraging Amazon Macie, we automatically analyze data stored in S3 buckets and identify sensitive data, such as personally identifiable information (PII) and sensitive information. This allows us to continuously monitor the storage status of sensitive data and establish a security management system that proactively detects and responds to potential data leaks. Please solve this module using the **ap-northeast-2** region.

Tasks

1. S3

We use S3 to manage files containing sensitive data. Create two prefixes in S3: */masked* and */incoming*. All given deployment files will be uploaded to the */incoming* prefix.
- S3 Bucket Name: **wsc2025-sensitive-<random four-letter>**

© Samsung Electronics

2. Macie

We use Amazon Macie to discover and protect sensitive data stored in S3. Macie must be able to accurately detect all sensitive information within provided files. The detection criteria are as follows: When creating a job, configure it to detect sensitive data only for objects with a ***masked*** prefix within your S3 bucket, and configure the job to run only once. When marking, copy the created job. If it occurs failure to perform this job properly may result in penalties that lose points.

- Job Name: ***wsc2025-sensor-job***

3. Lambda

To prevent sensitive data leakage, we use AWS Lambda to mask data. When a file is uploaded to ***incoming*** prefix path in an S3 bucket, the Lambda function automatically executes. It then masks sensitive information in the file and stores it in a path with the ***masked*** prefix. See the table below for examples of masking methods.

- Lambda Name: ***wsc2025-masking-start***

data	Raw data example (must be detected at Macie)	Masking data example (must be not detected at Macie)
names	Crystal White	Crystal *****
emails	davisjesus@example.org	d*********@example.org
phone_numbers	010-7658-5153	010-7658-****
ssns	887-07-7325	887-07-****
card_numbers	4468-6779-7028-4776	4468-6779-7028-****
uuids	665ef2db-cd63-4086-81e9-661ccaf8dd20	665ef2db-cd63-4086-81e9-**********

Module 3: Protect your API using WAF

Description

Many web applications perform database queries based on user-supplied values. If input validation is inadequate, malicious queries can be executed, potentially leading to SQL Injection (SQLi) attacks. SQLi is a serious security vulnerability that allows attackers to manipulate database queries to bypass authentication, leak data, alter/delete data, and more. Use a WAF to validate input and block malicious queries. Please solve this module using the **us-east-1** region.

Tasks

1. Bastion

For motion testing and marking, create an EC2 instance in the Default VPC, with the name tag "***waf-bastion***" Access to Session Manager of System Manager is required for marking; If it occurs failure to do so may result in penalty that lose points.

2. App Server

Create an App Server using EC2 in the default VPC. When running a Python application on the App Server using the given provided file (***main.py***), the web application must always run until marking time. And, It must assign a public IP address to allow Internet users.

- App Server Name: ***wsc2025-app-server***
- Instance Type: ***t3.small***

© Samsung Electronics

3. ELB

Create an ELB to provide an endpoint that can access the App Server created above.
- Load balancer name: ***wsc2025-waf-alb***
- Load balancer type: ***Application***
- Load balancer Scheme: ***Internet-facing***

4. WAF

Find vulnerabilities in provided application, configure rules for vulnerabilities, and ensure that a 403 Forbidden error occurs when exploiting the vulnerability.
- WAF Name: ***wsc2025-waf***

Ref. API

The application was developed in Python and built on an x86-based system. When run, it binds to TCP port 5000. The application endpoint and response values are listed in the table below.

Path	Method	Description	Query String Example
/login	GET	Authentication	name=admin&secret=x
/lookup	GET	Query by user id	id=1

Module 4: Secure your network using Network Firewall

Description

To effectively manage outgoing network traffic and strengthen security within a VPC environment, we'll configure a security system leveraging the AWS Network Firewall. The Network Firewall allows for granular control of VPC egress traffic and inspects network packets being forwarded to detect and block abnormal or malicious communications. Furthermore, by applying policy-based firewall rules to restrict communication to only permitted destinations and ports, we prevent internal resources from accessing external threats and unauthorized services. This strengthens the security of internal VPC resources and creates a secure network communication environment. Please solve this module using the **eu-west-1** region.

Tasks

1. VPC

The Egress VPC consists of a Public Subnet for internet connectivity, a Peering Subnet for communication with the App VPC, and a Firewall Subnet containing only the Network Firewall. The App VPC only creates a Private Subnet, and external communication is accomplished through the Firewall Subnet via the Egress VPC. The English letters after the subnet name indicate the AZ.

Ref. VPC Information

Name	Cidr
wsc2025-egress-vpc	10.0.0.0/16
wsc2025-app-vpc	172.16.0.0/16

© Samsung Electronics

Ref. Subnets information

Name	Cidr	VPC
wsc2025-egress-public-sn-a	10.0.0.0/24	wsc2025-egress-vpc
wsc2025-egress-public-sn-b	10.0.1.0/24	wsc2025-egress-vpc
wsc2025-egress-peering-sn-a	10.0.2.0/24	wsc2025-egress-vpc
wsc2025-egress-peering-sn-b	10.0.3.0/24	wsc2025-egress-vpc
wsc2025-egress-firewall-sn-a	10.0.4.0/24	wsc2025-egress-vpc
wsc2025-egress-firewall-sn-b	10.0.5.0/24	wsc2025-egress-vpc
wsc2025-app-private-sn-a	172.16.0.0/24	wsc2025-app-vpc
wsc2025-app-private-sn-b	172.16.1.0/24	wsc2025-app-vpc

2. Network Firewall

A Network Firewall is created in the firewall subnets of the Egress VPC, and the configured AWS Network Firewall systematically controls outgoing traffic to the external Internet through a multi-layered security strategy. This system implements a three-tiered defense mechanism.
First, the stateless rules engine blocks all ICMP (ping) protocol traffic, preventing network scanning and ping-based attacks. This serves as the first layer of defense against basic network layer (L3) security threats.
Second, the stateful rules engine blocks all external DNS queries using UDP/TCP port 53 to enhance DNS security. This enforces the use of only authorized internal DNS resolvers, preventing DNS hijacking attacks and data leaks. It also blocks indirect access via direct IP addresses or HTTPS encrypted traffic.
It is configured for high availability across two Availability Zones.
- Firewall Name: ***wsc2025-firewall***

3. Bastion

For testing and marking, create an EC2 instance in the App VPC, using the name tag "***app-bastion***". Access to SystemManager is required for marking. If occurs failure to do so may result in penalty that lose points.

Module 5: Serverless Inventory

Description

Skills Inc. wants to build a serverless inventory management system. They need to build a system that meets the following requirements without using separate computing services, using Step Functions, API Gateway, DynamoDB, and other tools. Computing services such as EC2, ECS, and Lambda cannot be used. Please solve this module using the **eu-west-1** region.

Tasks

1. DynamoDB

- Create a table of DynamoDB
  - Table name: inventory
- Insert records below before marking.
  - {“name”: “balance”, “value”: 1000}
  - {“name”: “stock”, “value”: 100}

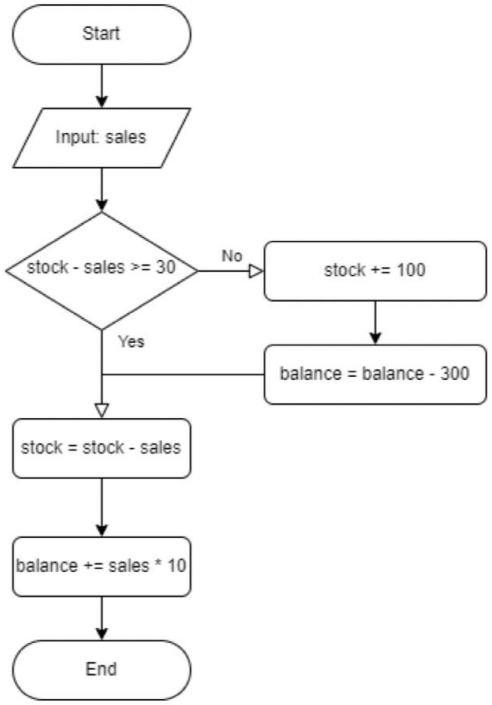
© Samsung Electronics

Ref. Scheme

Key name	Type	Description
name	String	PK
value	Number	

2. Step Functions

- Create a state machine of Step Functions
  - State machine name: inventory-state-machine
- State machine JSON input
  - ***sales***: 1-30 natural number
  - ex) {"sales": 15}
- You can refer that State machine flow



3. API Gateway

It has to provide the */inventory* API to query the contents of the inventory table in DynamoDB.

- API Type: REST API
- API name: ***inventory-rest-api***
- Stage name: ***v1***
- The */inventory* API only supports GET requests and the integration type must be specified as AWS Service Proxy.
- If a string starting with ***mysecret*** is received as the query string (*item*) of */inventory*, a response with HTTP status code 403 and body "**403 Forbidden**" is returned.

© Samsung Electronics

- an example of requests (the response must be excluded double quota)

Request	Response	Description
GET /inventory?item=balance	1000	
GET /inventory?item=stock	100	
GET /inventory?item=mysecret123	403 Forbidden	HTTP status code 403

- When marking, an HTTP request is made to like the following URL. It must be accessible from anywhere, such as the player's desktop or EC2.
- An example of requests URL: *https://5yb12xiewd.execute-api.us-west-1.amazonaws.com/v1/inventory?item=balance*

© Samsung Electronics